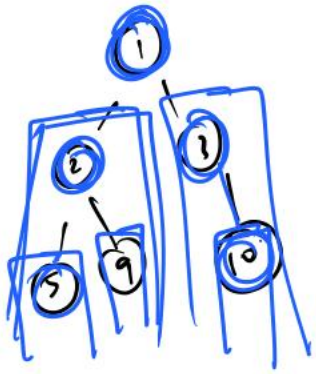
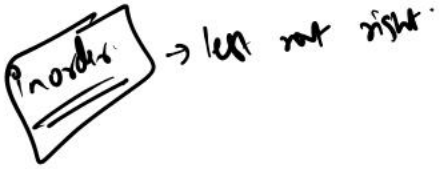
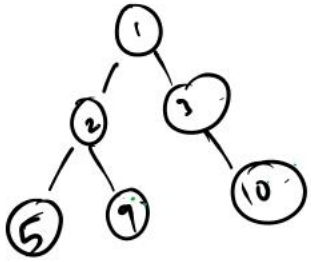
Day 2

Tuesday, 3 November 2026 10:18 AM



1 2 5 9 3 10



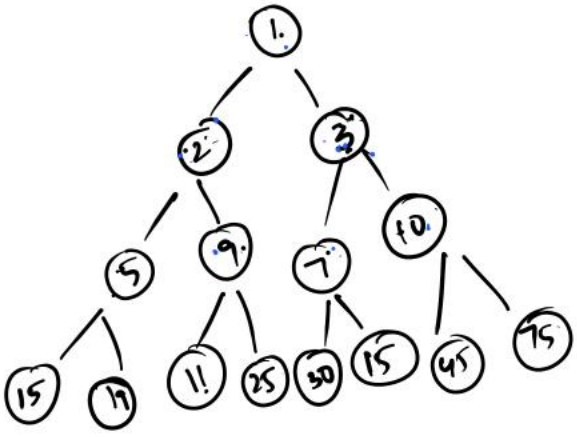
5 2 9 1 3 10

left right root.

↑

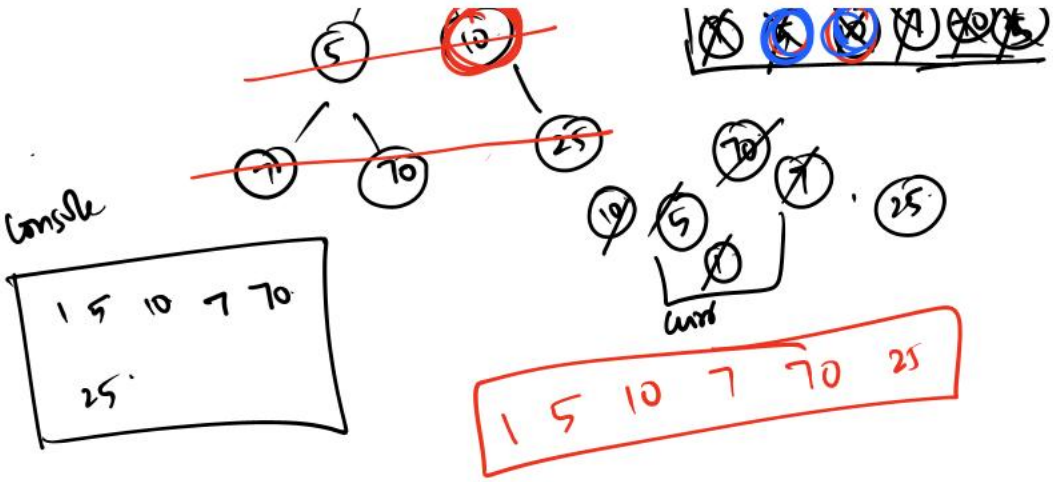
postorder.

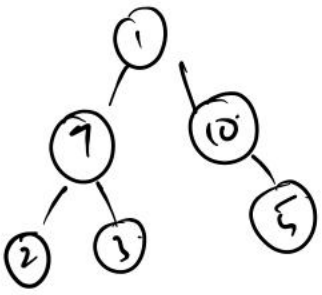
5 9 2 10 3 1



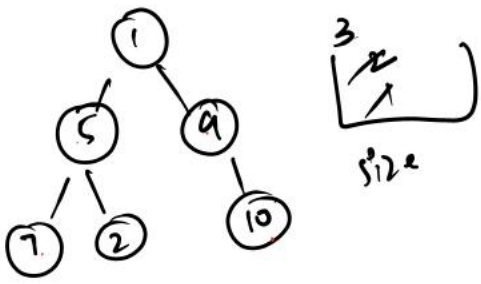






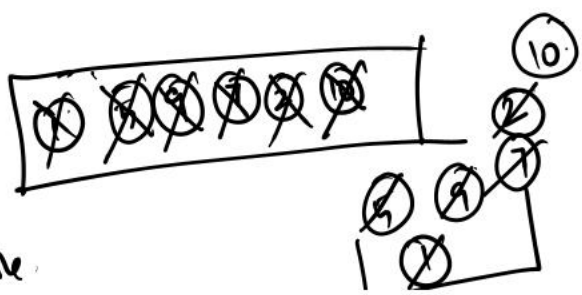



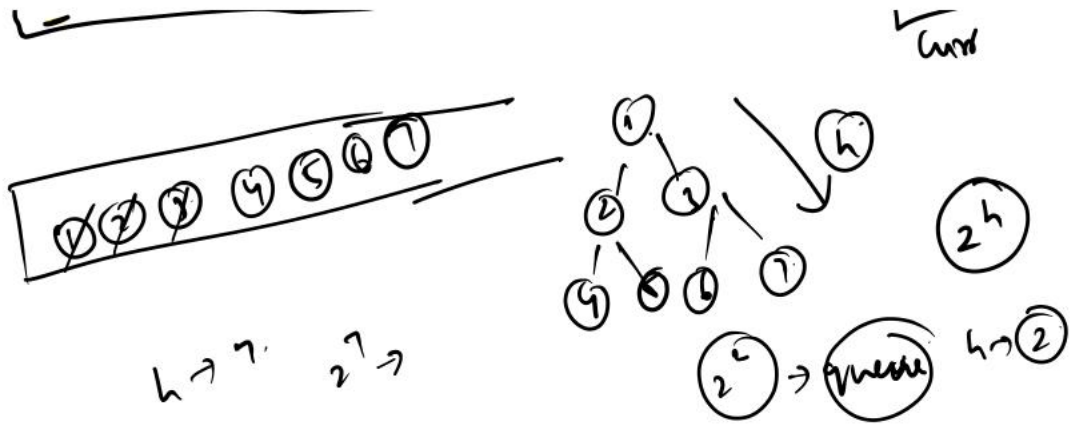
```
def levelOrder(root) :
    if root is None :
        return
    pendingNodes = deque()
    pendingNodes.append(root)
    while pendingNodes :
        size = len(pendingNodes)
        for x in range(size) :
            current = pendingNodes.popleft()
            print(current.data , end = ' ')
            if current.left :
                pendingNodes.append(current.left)
            if current.right :
                pendingNodes.append(current.right)
        print()
```









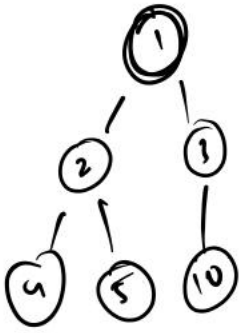


```python
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List
    [List[int]]:
        if root is None :
            return []
        pendingNodes = deque([root])
        output = []
        while pendingNodes :
            size = len(pendingNodes)
            currentLevel = []
            for x in range(size) :
                current = pendingNodes.popleft()
                currentLevel.append(current.val)
                if current.left :
                    pendingNodes.append(current.left)
                if current.right :
                    pendingNodes.append(current.right)
            output.append(currentLevel)
        return output
```

T.C → O(n)

S.C → O($2^h$)

h → height of tree

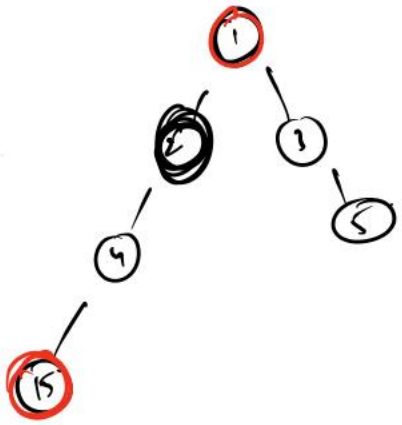


Complete Binary Tree

↓

(*) All levels should be filled except last level

(*) Left to Right



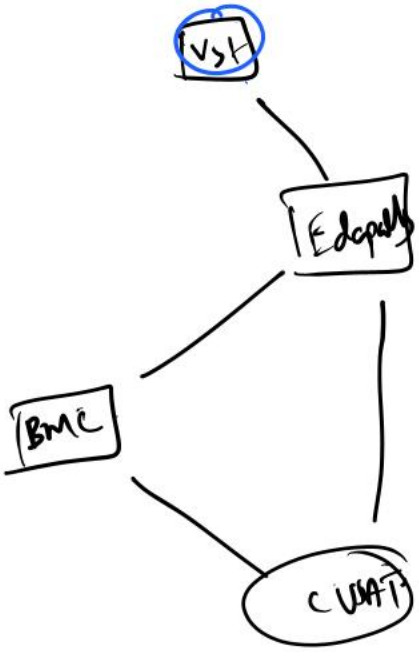

## Balanced Binary Tree.

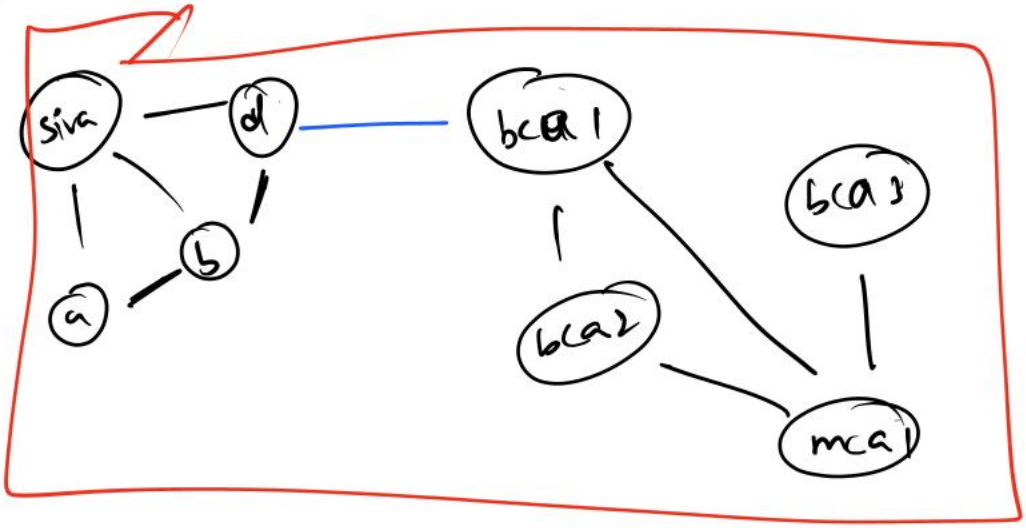
(*) $|\text{leftHeight} - \text{rightHeight}| \leq 1$

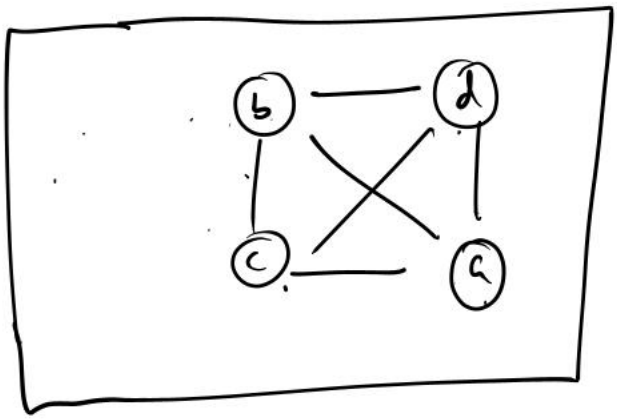
↓

(*) All Nodes in tree.

## Graphs





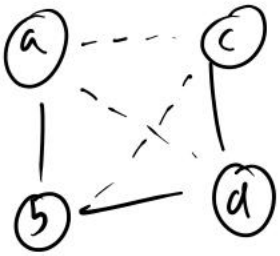

vertex. path

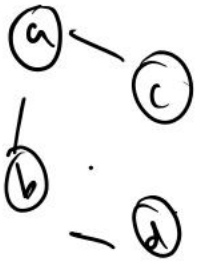
edges

degree → direct connections of a vertex.

connected graph. | connected components

complete graph.



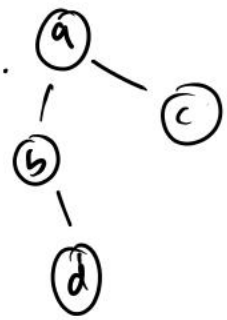

Tree

Spanning Tree.

↓

Minimum no of edges required to keep the graph as connected graph.

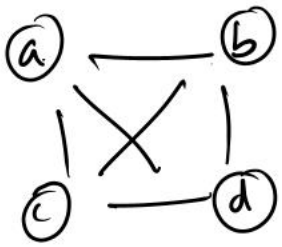
Tree




n → 4

edges → n-1

Complete Graph




n → 4

e → 6

$\frac{n(n-1)}{2}$

$n-1 \leq$ No of edges $\leq n^2$

Edge List

[(a,b)
(a,c)
(b,d)
⋮
]

① O(E)
B.C → O(n)
W.C → O(n²)

② O(E)
B.C → O(n)
W.C → O(n²)

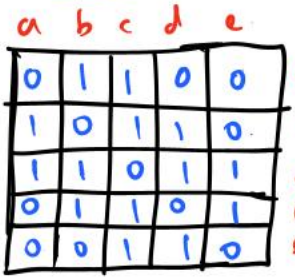
Adjacency List

{a: c,b
b: a, c, d
c: a, b, d, e
d: b, c, e.
e: c, d}

① O(n)

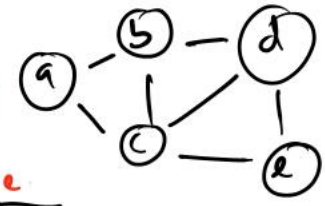
② O(n)

Adjacency Matrix




① O(1)

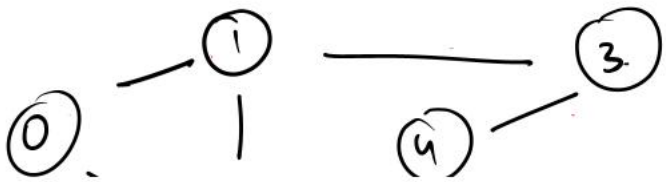
② O(n)




① v1 & v2 connected?

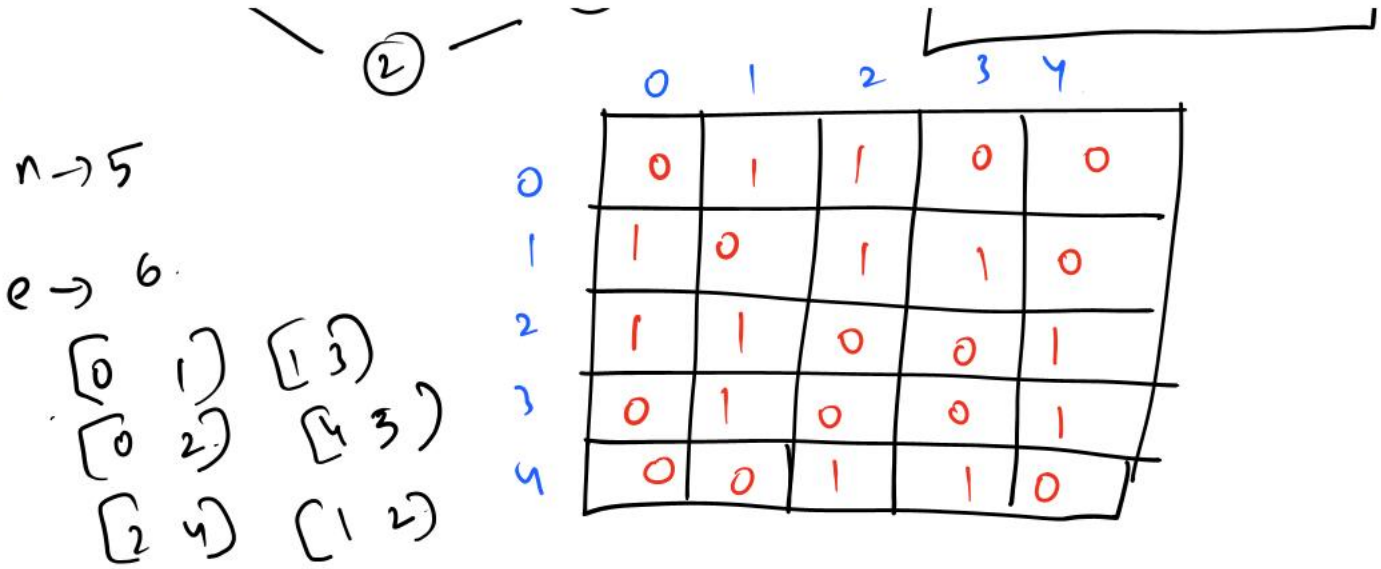
② Find neighbours of v1.

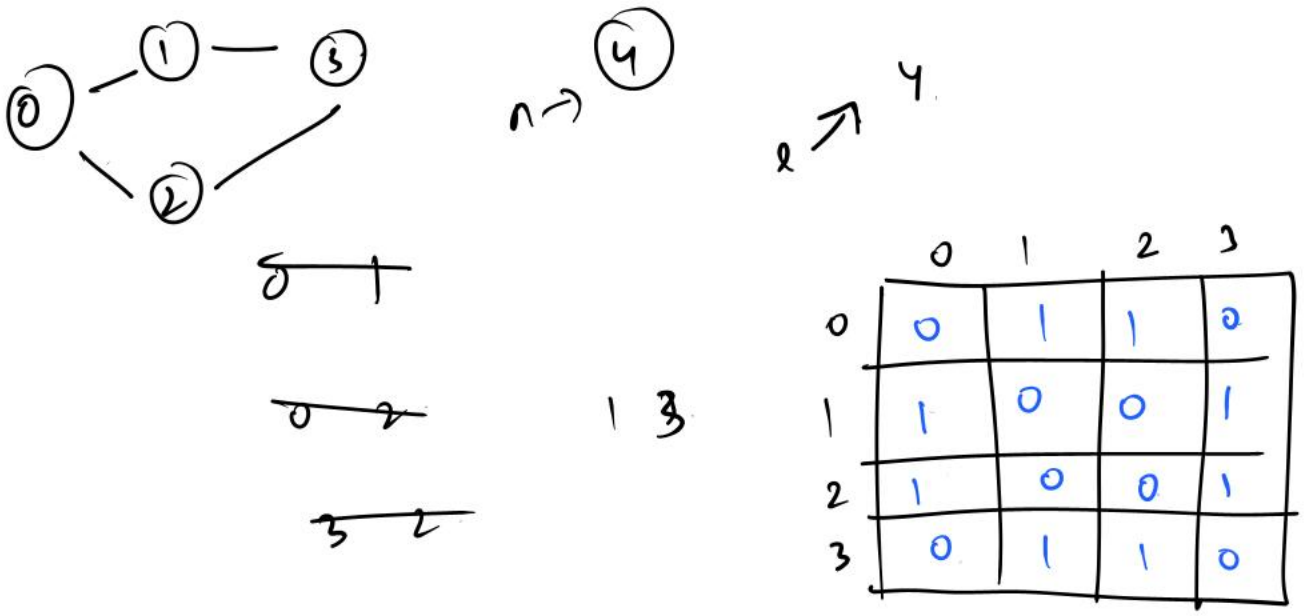
→ S.C → O(n²)




adjMatrix[v1][v2] = 1

adjMatrix[v2][v1] = 1




```python
def buildGraph() :
    n = int(input('Enter no of vertices : '))
    edges = int(input('Enter no of edges : '))
    adjMatrix = [[0] * n for _ in range(n)]
    for _ in range(edges) :
        v1 = int(input('Enter v1 : '))
        v2 = int(input('Enter v2 : '))
        adjMatrix[v1][v2] = 1
        adjMatrix[v2][v1] = 1
```

```
    return adjMatrix

adjMatrix = buildGraph()
for row in adjMatrix :
    for col in row :
        print(col , end = ' ')
    print()
```